



Tarea: Aprendizaje autónomo 1

Nombre: Mayky Cheyen Tituaña Lanchimba

Asignatura: Programación Orientada a Objetos

Profesor: ING. PALACIOS MOROCHO MILTON RICARDO

Carrera: Sistemas de la información

Proyecto: Sistema de Gestión de E-commerce

Repositorio GitHub: [https://github.com/maykyrojo123-bit/ecommerce-byMay-](https://github.com/maykyrojo123-bit/ecommerce-byMay-go)

go

Introducción	2
1. Objetivo del Código	3
2. Explicación de la Arquitectura y estructura del Proyecto	3
3. Documentación de Paquetes	4
4. Carpetas, Ficheros y su Función	4

5. Planificación de Persistencia y Base de Datos.....	10
6. Diagrama de Clases UML.....	11

Introducción

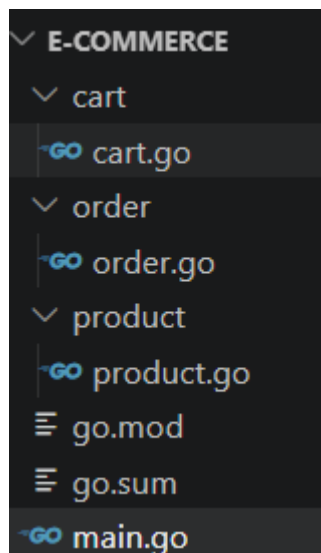
El comercio electrónico (*e-commerce*) se ha convertido en una de las arquitecturas más demandadas, requiriendo sistemas backend de alta velocidad, seguros y con una óptima gestión de memoria, el informe documenta el desarrollo y la planificación de la Etapa 1 del Sistema de Gestión de E-commerce enfocado en el sector mobiliario.

A través de este proyecto, se aplican de forma práctica los fundamentos de la Programación Orientada a Objetos (POO) en Go, tales como la abstracción mediante estructuras y el encapsulamiento a nivel de paquetes, incorporando menús interactivos gobernados por flujos iterativos continuos (for), toma de decisiones analíticas (switch-case), control flexible de carritos de compras mediante arreglos dinámicos (Slices) y la gestión eficiente de inventarios en memoria a través de tablas de búsqueda de pares clave-valor (Maps).

1. Objetivo del Código

El objetivo de este desarrollo es construir la lógica de negocio (backend) de un Sistema de Gestión de E-commerce, el código está diseñado para simular el proceso completo de compra de una tienda de mobiliario: permite al usuario ingresar las cantidades deseadas, realiza el cálculo matemático de subtotales, inyecta reglas de facturación (como el 15% de IVA) y gestiona la reducción segura del inventario. Para esto, vamos a usar el lenguaje de programación GO

2. Explicación de la Arquitectura y estructura del Proyecto



Como se observa en la imagen, el proyecto no se programó en un solo archivo, sino que se estructuró utilizando una arquitectura modular por paquetes. En Go, la Programación Orientada a Objetos y el encapsulamiento se gestionan a nivel de directorios.

Esta estructura cumple el principio de alta cohesión y bajo acoplamiento. Cada carpeta actúa como un espacio de nombres aislado, protegiendo los datos sensibles (como el stock del inventario) para que no puedan ser alterados.

3. Documentación de Paquetes

Para el desarrollo de este backend, se identificaron e incorporaron los siguientes paquetes, divididos entre la librería estándar y dependencias de terceros:

3.1 Paquetes Nativos (Librería Estándar de Go):

fmt: Utilizado para la interacción mediante la Interfaz de Línea de Comandos (CLI), formateo de cadenas de texto y captura de entradas del usuario mediante punteros (`fmt.Scan`).

os: Implementado para gestionar las señales del sistema operativo, permitiendo un cierre limpio y seguro de la aplicación (`os.Exit(0)`).

time: Incorporado en el módulo `order` para estampar la marca de tiempo (timestamp) exacta en la que se procesa el *checkout* de la compra.

3.2 Paquetes de Terceros (Externos)

Paquete: `github.com/google/uuid` (Versión v1.6.0)

Se eligió e incorporó esta librería de Google para generar Identificadores Únicos Universales (UUID). Se utilizó UUID para garantizar identificadores únicos para productos y órdenes.

4. Carpetas, Ficheros y su Función

4.1 Carpeta `product` / Fichero `product.go`:

Principal: `main.go`

```
main.go > main
1 package main
```

Contiene el punto de Entrada función principal func main().

Funciones y lógica aplicada:

1. **Inicialización:** Instancia los objetos de los productos y los guarda dentro de una estructura de **Map** (catalogo), que funciona como base de datos temporal en memoria.
2. **Interfaz de Usuario (CLI):** Utiliza la librería nativa fmt (fmt.Println y fmt.Print) para dibujar un menú interactivo en la terminal con 4 opciones.
3. **Flujo Iterativo Permanente:** Implementa un bucle **for infinito** para asegurar que el menú se repita constantemente y el sistema no se cierre tras realizar una sola acción.
4. **Control de Decisiones:** Utiliza una estructura **switch-case** para evaluar la opción numérica del usuario. Captura los datos del teclado mediante **punteros** (fmt.Scan(&opcion)).

La opción 1 despliega el catálogo recorriendo el Map; la opción 2 busca de forma segura si el producto existe y lo añade al carrito; la opción 3 ejecuta el cierre de la compra (*checkout*) e imprime la factura electrónica final usando os.Exit(0) para un cierre limpio.

```

30     for {
31         fmt.Println("\n=====")
32         fmt.Println("    UIDE DESIGN    ")
33         fmt.Println("=====")
34         fmt.Println("1. Ver Catálogo de Productos y Stock")
35         fmt.Println("2. Agregar Muebles al Carrito")
36         fmt.Println("3. Procesar Compra (checkout)")
37         fmt.Println("0. Salir del Sistema")
38         fmt.Print("👉 Seleccione una opción: ")
39
40         var opcion int
41         fmt.Scan(&opcion)
42
43         //estructura switch-case
44         switch opcion {
45             case 1:
46                 fmt.Println("\n--- CATÁLOGO EN BODEGA (Mapeado) ---")
47                 for clave, prod := range catalogo {
48                     fmt.Printf("• Código para comprar: [%s] | %s - Precio: $%.2f | Stock: %d unids.\n",
49                         clave, prod.Name, prod.Price, prod.GetStock())
50                 }

```

```

    UIDE DESIGN
=====
1. Ver Catálogo de Productos y Stock
2. Agregar Muebles al Carrito
3. Procesar Compra (Checkout)
0. Salir del Sistema
👉 Seleccione una opción: 3

```

4.2 Inventario: product/product.go

Contiene el paquete product. No requiere importaciones externas complejas ya que modela datos nativos.

Guarda los atributos esenciales de un artículo: ID (string), Name (string), Price (float64) y stock (int).

NewProduct(): Funciona como el **constructor** del objeto, permitiendo inicializar nuevos muebles con sus datos base.

GetStock(): Un método receptor que expone el stock de manera segura (función *Getter*).

ReduceStock(): La función estrella de **encapsulamiento**. Recibe una cantidad y, mediante un condicional if, valida que existan unidades suficientes. Si todo es correcto, resta el stock directamente en la dirección de memoria usando un puntero receptor (p *Product).

```
1  package product
2
3  import "errors"
4
5  type Product struct {
6      id      string
7      Name    string
8      Price   float64
9      stock   int
10 }
11
12
13 func NewProduct(id string, name string, price float64, stock int) *Product {
14     return &Product{id: id, Name: name, Price: price, stock: stock}
15 }
16
17 func (p *Product) GetID() string {
18     return p.id
19 }
20
21 func (p *Product) GetStock() int {
22     return p.stock
23 }
24
25 func (p *Product) ReduceStock(quantity int) error {
```

4.3 Intención de Compra: cart/cart.go

Contiene el paquete cart e importa internamente el paquete ecommerce-poo/product para poder asociar los artículos al carrito.

Guarda dos estructuras críticas para la agregación de datos:

CartItem: Guarda el objeto Product y la Quantity que el usuario decidió comprar.

Cart: Guarda una colección dinámica de elementos utilizando un Slice de tipo []CartItem.

AddItem(): Recibe el producto y la cantidad deseada. Utiliza la función nativa `append()` para empujar el nuevo elemento al final del Slice dinámico, permitiendo que el carrito crezca ilimitadamente a medida que el cliente agrega más muebles.

```
cart > cart.go > (*Cart).CalculateSubtotal
1  package cart
2
3  import "ecommerce-poo/product"
4
5  type CartItem struct {
6      Product product.Product
7      Quantity int
8  }
9
10 type Cart struct {
11     Items []CartItem
12 }
13
14 func (c *Cart) AddItem(p product.Product, qty int) {
15     item := CartItem{Product: p, Quantity: qty}
16     c.Items = append(c.Items, item)
17 }
18
19 func (c *Cart) CalculateSubtotal() float64 {
20     var subtotal float64
21     for _, item := range c.Items {
22         subtotal += item.Product.Price * float64(item.Quantity)
23     }
24     return subtotal
25 }
26
```

4.4 Cierre y Reglas: order/order.go

Contiene el paquete `order`. Importa los paquetes internos `product` y `cart`, además de la librería nativa `time` para auditoría temporal.

Guarda:

1. `EcuadorTax`: Estructura que guarda la tasa impositiva (Rate: 0.15) aplicando principios de polimorfismo estratégico.

2. Order: Estructura final que guarda los resultados de la transacción: ID de la factura, la fecha exacta de creación (CreatedAt), la copia de los ítems cobrados, y los valores numéricos finales (Subtotal, Tax y Total).

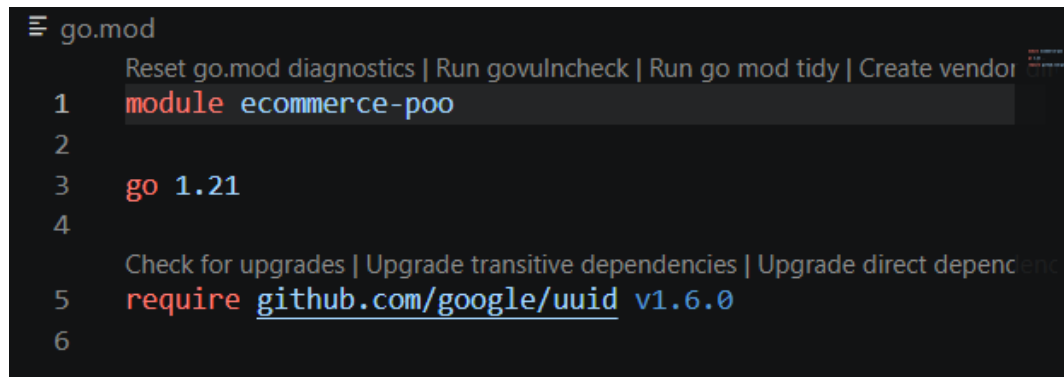
ProcessCheckout(): Recibe el carrito lleno, ejecuta un bucle for para sumar los precios de todos los productos y calcular el Subtotal. Luego aplica la multiplicación del IVA (15%) y calcula el total definitivo, estampando la marca de tiempo exacta del servidor.

```
20  type Order struct {
21      ID      string
22      CartItems []cart.CartItem
23      Subtotal float64
24      Tax      float64
25      Total    float64
26      CreatedAt time.Time
27  }
28
29  func (o *Order) ProcessCheckout(id string, c cart.Cart, taxCalc TaxCalculator) {
30      o.ID = id
31      o.CartItems = c.Items
32      o.Subtotal = c.CalculateSubtotal()
33      o.Tax = taxCalc.CalculateTax(o.Subtotal)
34      o.Total = o.Subtotal + o.Tax
35      o.CreatedAt = time.Now()
```

4.5 Configuración del Módulo: go.mod

Contiene el manifiesto del proyecto.

Función: Declara formalmente el nombre del módulo raíz (ecommerce-poo), define que el compilador debe usar las reglas sintácticas de la versión go 1.21 y registra la cláusula require que obliga al sistema a descargar y usar la librería de seguridad github.com/google/uuid de forma mandatoria.



```

1 module ecommerce-poo
2
3 go 1.21
4
5 require github.com/google/uuid v1.6.0
6

```

4.6 Registro de Integridad Criptográfica: go.sum

Este archivo se generó de forma totalmente automática en el momento en que ejecutamos el comando `go run main.go` tras haber agregado la librería `google/uuid`.

Función: El backend necesita garantizar que el código que descargamos de internet sea seguro. `go.sum` guarda una lista exacta de **hashes criptográficos (firmas digitales)** de la librería de Google. Si alguien intentara modificar el código de esa librería en internet para meternos un virus, el archivo `go.sum` detectaría que los hashes no coinciden y bloquearía el sistema inmediatamente por seguridad.



```

1 github.com/google/uuid v1.6.0 h1:NiVaJD0MsjH48n1jAhLSgzrAzyIHgr+hNrb57e+94F0=
2 github.com/google/uuid v1.6.0/go.mod h1:T1yPZe4MgqvfeYDBFedMoGGpEw/LqOeaOT+nhxU+yHo=
3

```

5. Planificación de Persistencia y Base de Datos

En esta Etapa el sistema opera utilizando la memoria temporal de la máquina a través de estructuras dinámicas (Maps y Slices), para la evolución del proyecto, se ha definido que el sistema migrará hacia una **Base de Datos Relacional (SQL)**, específicamente utilizando **MySQL**, debido a que su adopción con los requerimientos y proyectos del segundo semestre

6. Diagrama de Clases UML

